

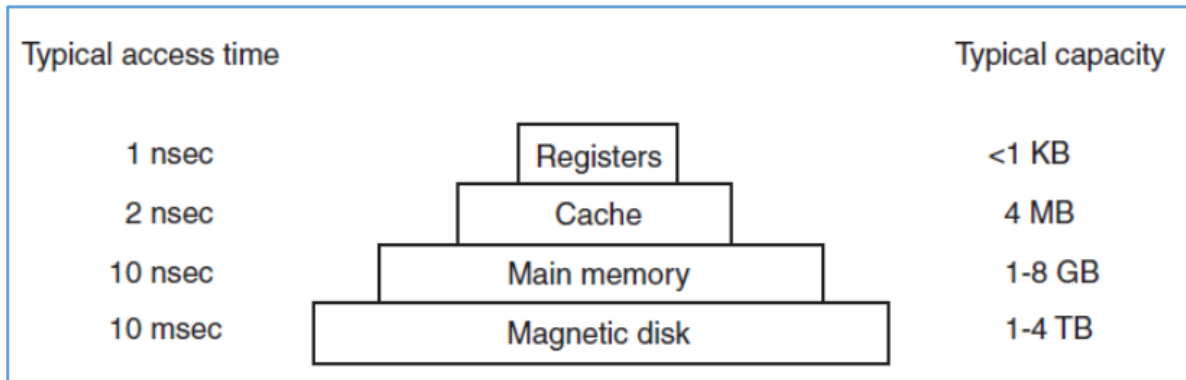
UJIAN TENGAH SEMESTER GENAP TAHUN AKADEMIK 2021/2022

IF260 - Sistem Operasi (Teori)

Kafijaya (00000061651)

Nomor 1

Jelaskan mengapa memori dalam komputer memiliki hirarki seperti gambar berikut dan jelaskan kaitannya terhadap desain prinsipal suatu sistem operasi!



Jawab:

Memori memiliki hierarki seperti gambar dikarenakan berfungsi untuk mengkategorikan jenis-jenis memori kedalam kecepatan memori, besar kapasitas memori, dan harga memori dimana semakin tinggi jenis memori berada dalam hierarki, maka semakin kecil kapasitas memori namun semakin cepat dan semakin tinggi harga memori. Sehingga urutannya ialah Registers, Cache, Main memory, dan Magnetic disk.

Kaitan hierarki memori kedalam desain principal memori adalah semakin tinggi hierarki memori maka jenis memori akan semakin menjadi bagian dari system. Dengan memori di hierarki bagian atas menjadi bagian dari embedded system dan bagian bawah hierarki merupakan eksternal system.

Nomor 2

- a. Jelaskan apa yang dimaksud dengan proses zombie dan proses orphan?

Jawab:

- Zombie Process terjadi karena adanya child process yang di exit namun parent processnya tidak tahu bahwa child process tersebut telah di terminate, misalnya disebabkan karena putusnya network. Sehingga parent process tidak merelease process yang masih digunakan oleh child process tersebut walaupun process tersebut sudah mati.
- Orphan Process adalah sebuah proses yang ada dalam komputer dimana parent process telah selesai atau berhenti bekerja namun proses child sendiri tetap berjalan. Salah satu contohnya adalah saat suatu proses berjalan sangat lama dalam menyelesaikan tugasnya tanpa perhatian sang user.

- b. Perhatikan potongan program di bawah ini, apakah program dapat menghasilkan proses zombie atau proses orphan?

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    int x=100;
    int y=50;
    int i, z;
    if(fork() == 0){
        for(i=0; i<20; i++) x++;
    }
    else{
        for(i=0; i<20; i++) y++;
    }
    printf("z = %d\n", x+y);
    return 0;
}
```

Jawab:

Program dalam soal ini bisa menghasilkan proses orphan maupun proses zombie karena dalam proses fork, terdapat int yang sama sehingga dapat terjadi saling timpa satu sama lain. Dan program dapat menghasilkan zombie ataupun orphan tergantung pada variabel yang ditimpa.

Zombie process : Hal ini dapat terjadi bila child process berjalan terlebih dahulu. Ketika child process selesai menjalankan eksekusinya, kemudian child process akan menunggu terlebih dahulu parent process berjalan dan menyelesaikan eksekusinya. Sehingga zombie process terjadi karena child process masih terdaftar pada entry process walaupun telah selesai menjalankan eksekusinya.

Orphan process : Hal ini dapat terjadi bila parent process berjalan terlebih dahulu. Walaupun parent telah selesai menjalankan eksekusinya dan processnya berakhir, child process masih berjalan hingga selesai menjalankan eksekusinya.

- c. Apakah hasil eksekusi program tersebut dan jelaskan mengapa?

Jawab:

Output:

z = 170

z = 170

Mengapa:

Pertama kali jalan, value dari fork() bukan == 0, maka masuk fungsi else, ketika fungsi else telah dijalankan sampai selesai akan jalan return 0, maka fork() langsung berubah value nya menjadi 0, dan if(fork() == 0) berjalan sampai habis dan menghasilkan tampilan seperti diatas.

Nomor 3

- a. Jelaskan apa yang dimaksud dengan Critical Region?

Jawab:

Critical Region adalah bagian program yang memiliki kondisi rentan eror karena terdapat data atau resources yang diakses secara bersamaan oleh beberapa process atau thread sehingga dapat menghasilkan hasil yang non-deterministic atau inconsistent atau yang tidak diinginkan seperti deadlock, starvation. Diperlukan mutual exclusion yang menjadi suatu kondisi supaya tidak ada proses yang mempengaruhi suatu data atau resource yang sedang dipakai oleh suatu proses.

- b. Apakah terdapat Critical Region pada program berikut? Jelaskan!

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

long x = 0;

void *Proses1(void *arg){
    int i;
    for(i=0; i < 1500000; i++) x++;
    return NULL;
}

void *Proses2(void *arg){
    int i;
    for(i=0; i < 1500000; i++) x++;
    return NULL;
}

int main()
{
    pthread_t thrd1, thrd2;
    pthread_create(&thrd1, NULL, Proses1, NULL);
    pthread_create(&thrd2, NULL, Proses2, NULL);
    pthread_join(thrd1, NULL);
    pthread_join(thrd2, NULL);
    printf("x = %ld\n", x);
    return 0;
}
```

Jawab:

Critical Region yang ada pada program adalah x yang sama-sama dipanggil di thread1(*Proses1) dan thread2(*Proses2).

- c. Ada berapa proses dan thread yang tercipta dari eksekusi program di atas? Jelaskan!

Jawab:

Pada program tersebut terdapat 1 Proses dan 3 Thread (main, *Proses1, *Proses2) yang tercipta dari eksekusi program tersebut.

- d. Jelaskan apa yang dimaksud dengan Race Condition?

Jawab:

Race Condition adalah suatu kondisi dimana dua atau lebih proses atau thread saling berkejaran karena mengakses suatu sumber daya secara bersama-sama sehingga dapat menimbulkan masalah. Race condition ini terjadi apabila hasil dari suatu operasi ditentukan oleh urutan dari proses pada data yang di-share.

- e. Apakah program di atas menimbulkan Race Condition?

Jawab:

Program diatas terdapat race condition pada saat variabel 'x' di-update di masing-masing thread (thread1 dan thread2) karena nilai dari variabel 'x' akan saling berkejaran di tiap iterasinya, menambah value yang membuatnya tidak konsisten dan hasilnya selalu lebih dari 1500000.

Nomor 4

Diketahui 6 buah batch jobs dengan Arrival Time dan Run Time seperti table di bawah: Tentukanlah average turnaround time dan average waiting time dengan menggunakan algoritma scheduling di bawah ini !

Jobs	Arrival Time	Run Time
J1	0	10
J2	2	6
J3	3	3
J4	5	4
J5	7	7
J6	8	1

1. Shortest remaining time next

Jawab:

J1	J2	J3	J4	J5	J6	J3	J4	J2	J5	J1
----	----	----	----	----	----	----	----	----	----	----

JOBS	Finish Time	Turnaround Time	Waiting Time
J1	31	31	21
J2	17	15	9
J3	10	7	4
J4	12	7	3
J5	23	16	9
J6	9	1	0

Avg Waiting Time = $(21+9+4+3+9+0)/6 = 7.67s$

Avg Turnaround Time = $(31+15+7+7+16+1)/6 = 12.83s$

2. Round Robin dengan Quantum time = 1

Jawab:

0	1	7	10	14	21	22	30
J1	J2	J3	J4	J5	J6	J1	

PROSES	WAITING
J1	22
J2	1
J3	7
J4	10
J5	14
J6	21

Avg Waiting = $(22+1+7+10+14+21)/6 = 12.5s$

Avg Turn Around = $(30+7+10+14+21+22)/6 = 17.3s$